

P4-03 单元测试代码与运行记录

项目： CampusHub
阶段： P4 编码开发
对应交付物： 3. 单元测试代码

1. 文档目的

本文档记录 P4 阶段补充的单元测试代码、覆盖范围和运行结果。测试重点不平均铺开，而是围绕 **P4-04 Bug 修复日志** 中出现过的真实问题补防回归测试。

2. 后端单元测试文件

文件	测试对象	重点覆盖的 P4 Bug 类型
OrderServiceTest.java	OrderService	订单状态流转、服务方取消申请、发布方同意后退回待接单、评价重复提交、信用分边界
AdminServiceTest.java	AdminService	用户状态管理、后台任务/订单/举报处理
AuthServiceImplTest.java	AuthServiceImpl	注册验证码发送场景区分，避免已注册邮箱再次走注册发码流程
EmailServiceTest.java	EmailService	验证码邮件 log/smtp 模式，避免本地无 SMTP 时阻塞开发联调
FileServiceTest.java	FileService	上传文件归属和用途校验，防止任务图、聊天图、举报证据互相复用
NotificationFactoryTest.java	NotificationFactory	订单取消申请/处理结果通知内容生成
NotificationServiceTest.java	NotificationService	订单动作通知持久化
ReportServiceTest.java	ReportService	举报证据与举报记录持久化关联
TaskServiceTest.java	TaskService	非法任务分类返回业务错误而不是 500
UserServiceImplTest.java	UserServiceImpl	信用分读取时限制在 0-100

3. 前端单元测试文件

文件	测试对象	重点覆盖的 P4 Bug 类型
frontend/src/utils/assets.test.ts	resolveAssetUrl	上传图片返回 /uploads/...、blob:、data:、绝对 URL 时的资源地址处理

文件	测试对象	重点覆盖的 P4 Bug 类型
frontend/src/utils/orderStatus.test.ts	compareOrdersByStatus	“我的订单”和任务状态排序，进行中优先，已取消最后

前端测试使用 Vitest，配置文件为：

```
frontend/vitest.config.ts
```

4. 本次测试发现并修复的问题

补充单元测试时发现一个与 P4-04 订单取消流程相关的真实回归点：

```
发布方取消订单或同意服务方取消申请后，订单状态退回 PENDING_CONFIRM，  
但后端未同步清空 service_provider_id。
```

已修复位置：

```
backend/src/main/java/com/campusHub/service/OrderService.java
```

修复内容：

- 1. 发布方直接取消时，内存对象和数据库更新同时清空 `serviceProviderId`。
- 2. 发布方同意服务方取消申请时，内存对象和数据库更新同时清空 `serviceProviderId`。
- 3. 保留订单状态日志，继续用于审计和问题追踪。

5. 运行命令

后端单元测试：

```
cd backend  
mvn -q -  
Dtest=AdminServiceTest,OrderServiceTest,AuthServiceImplTest,EmailServiceTest,FileS  
erviceTest,NotificationFactoryTest,NotificationServiceTest,ReportServiceTest,TaskS  
erviceTest,UserServiceImplTest test
```

前端单元测试：

```
cd frontend
```

```
npm run test
```

6. 本地验证结果

范围	命令	结果
后端单元测试	<code>mvn -q -Dtest=... test</code>	已通过
前端单元测试	<code>npm run test</code>	已通过，2 个测试文件、3 个用例

说明：本地后端首次运行需要 Maven 下载依赖；前端首次运行需要 `npm install` 同步 `package-lock.json`。

7. 覆盖率说明

当前已经为每个后端 Service 相关类补充了至少一个单元测试，并为前端关键防回归逻辑补充 Vitest 测试。但项目尚未接入 JaCoCo，因此仍不能自动给出精确覆盖率百分比。

后续如需严格证明“核心模块覆盖率 $\geq 60\%$ ”，建议：

- 1. 在后端 `pom.xml` 接入 JaCoCo。
- 2. 在 GitLab CI 中生成覆盖率报告。
- 3. 继续补充 `AuthServiceImpl`、`TaskService`、`ReportService` 的更多异常分支测试。